

# Presentación de las métricas encontradas en DentalCare

Miguel Monsalve Osorio

## 1. Introducción

El caso de citas de DentalCare presenta deficiencias en su diseño al violar los cinco principios SOLID y concentrar un número elevado de responsabilidades en sus componentes principales. En este informe se presenta el diagnóstico detallado de estas violaciones, la línea base de métricas del sistema y una comparación de antes y después de los resultados esperados.

## Repositorio

[MiguelMonsalve1/ArquitecturaSoftware-Practica1](https://github.com/MiguelMonsalve1/ArquitecturaSoftware-Practica1)

## 2. Diagnóstico SOLID

### Matriz de violaciones

| Principio | Dónde (archivo y líneas)                                                                    | Qué se viola                                                                                                                             | Consecuencia                                                                         |
|-----------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| SRP       | GestorCitasOdontologicas.cs: 7-10, 12-73, 75-93 y 95-97                                     | Una sola clase valida disponibilidad, calcula tarifas, crea la cita, la guarda, notifica, aplica multas y lleva la contabilidad.         | 11 responsabilidades que responden a tres actores distintos.                         |
| OCP       | GestorCitasOdontologicas.cs: 22-37, 40-47 y 79-83.<br>NotificacionServicio.cs: 12-22        | Hay varios if/else con códigos numéricos para especialidad, convenio y multa. Los canales de notificación están fijos.                   | Cada especialidad, convenio o canal nuevo obliga a modificar código que ya funciona. |
| LSP       | NotificacionServicio.cs: 12 y 15-17.<br>GestorCitasOdontologicas.cs: 22-37, 81 y 89         | El método EnviarEmailYSms recibe un mensaje y lo ignora. Además, el código pregunta por el código de tipo en lugar de usar polimorfismo. | Cuando se cancela una cita, el paciente recibe "Su cita quedó programada".           |
| ISP       | NotificacionServicio.cs: 12-22.<br>GestorCitasOdontologicas.cs: 5-97. Program.cs: 8 y 48-49 | Un solo método obliga a enviar correo y SMS a la vez, y el gestor                                                                        | Quien solo necesita SMS o solo reportes queda atado a                                |

|     |                                                                                                                                    |                                                                                                           |                                                                                                                                            |
|-----|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|     |                                                                                                                                    | expone agenda, cancelación y reportes a cualquier cliente.                                                | SMTP, Twilio y SQL Server.                                                                                                                 |
| DIP | GestorCitasOdontologicas.cs: 1, 7 y 8. SqlServerEjecutor.cs: 3, 10, 13 y 27. NotificacionServicio.cs: 3, 10–11 y 14. Program.cs: 8 | La lógica de negocio crea con new las clases de infraestructura y no existe ninguna interfaz entre ellas. | No se puede probar nada sin una base de datos y un servidor de correo reales, y cambiar de proveedor obliga a tocar las reglas de negocio. |

### 3. Línea base cuantitativa (Baseline)

Todas las métricas de esta sección se calculan sobre la clase monolítica GestorCitasOdontologicas.

#### a. LCOM96b

$$LCOM96b = \frac{1}{a} * \sum_{j=1}^a \left[ \frac{m - \mu(A_j)}{m} \right]$$

**Paso 1.** Identificar los atributos (a = 4)

A<sub>1</sub> = \_db

A<sub>2</sub> = \_notificador

A<sub>3</sub> = \_totalRecaudadoMes

A<sub>4</sub> = \_totalCitasCanceladas

**Paso 2.** Identificar los métodos (m = 4)

M<sub>1</sub> = AgendarCita

M<sub>2</sub> = CancelarCita

M<sub>3</sub> = ObtenerTotalRecaudado

M<sub>4</sub> = ObtenerTotalCanceladas

**Paso 3.** Contar cuántos métodos usan cada atributo

| Atributo (A <sub>j</sub> ) | Métodos que lo usan                              | μ(A <sub>j</sub> ) | (m – μ(A <sub>j</sub> )) / m |
|----------------------------|--------------------------------------------------|--------------------|------------------------------|
| A <sub>1</sub> = _db       | AgendarCita (línea 68) y CancelarCita (línea 88) | 2                  | (4 – 2) / 4 = 0.50           |

|                                       |                                                                |   |                      |
|---------------------------------------|----------------------------------------------------------------|---|----------------------|
| $A_2 = \text{\_notificador}$          | AgendarCita (línea 69) y<br>CancelarCita (línea 89)            | 2 | $(4 - 2) / 4 = 0.50$ |
| $A_3 = \text{\_totalRecaudadoMes}$    | AgendarCita (línea 71) y<br>ObtenerTotalRecaudado (línea 95)   | 2 | $(4 - 2) / 4 = 0.50$ |
| $A_4 = \text{\_totalCitasCanceladas}$ | CancelarCita (línea 91) y<br>ObtenerTotalCanceladas (línea 97) | 2 | $(4 - 2) / 4 = 0.50$ |

**Paso 4.** Reemplazar en la fórmula

$LCOM96b = (1/4) \times [(4 - 2)/4 + (4 - 2)/4 + (4 - 2)/4 + (4 - 2)/4]$

$LCOM96b = (1/4) \times [0.50 + 0.50 + 0.50 + 0.50]$

$LCOM96b = (1/4) \times 2.00$

$LCOM96b = 0.50$

**Interpretación:** LCOM96b va de 0 a 1. Un 0 significa que todos los métodos trabajan con todos los atributos, y un 1 que no comparten nada. Con 0.50, cada atributo lo usa solo la mitad de los métodos: agendar y cancelar trabajan con la base de datos y el notificador, mientras que cada contador vive en su propio flujo. Es la forma numérica de decir que dentro de una sola clase conviven tareas que no tienen mucho que ver entre sí.

## b. Acoplamiento $C_a$ y $C_e$

$C_a$  (acoplamiento aferente) cuenta cuántas clases dependen de GestorCitasOdontologicas.  $C_e$  (acoplamiento eferente) cuenta de cuántas clases depende ella.

**$C_a = 1$**  (conexiones entrantes)

Program: crea el gestor con new GestorCitasOdontologicas() y llama a sus métodos. Es la única clase del proyecto que lo usa.

**$C_e = 5$**  (conexiones salientes)

SqlServerEjecutor: clase concreta de infraestructura para SQL Server.

NotificacionServicio: clase concreta de infraestructura para correo y SMS.

Cita: entidad del modelo.

Paciente: entidad del modelo.

Odontologo: entidad del modelo.

**Interpretación:** el gestor depende de cinco clases y solo una depende de él. Más que la cantidad, preocupa de quién depende: dos de esas cinco son clases concretas de infraestructura. Por eso cualquier cambio en la base de datos o en el envío de correos llega directo a la clase que tiene las reglas del negocio.

### c. Inestabilidad (I), Abstracción (A) y Distancia (D)

$$I = \frac{C_e}{C_e + C_a},$$

$$A = \frac{\sum m_a}{\sum m_c}, \quad D = |A + I| - 1$$

#### **Inestabilidad**

$$I = C_e / (C_e + C_a)$$

$$I = 5 / (5 + 1)$$

$$I = 5 / 6$$

$$I = 0.83$$

**Interpretación:** la inestabilidad va de 0 (estable, difícil de mover) a 1 (inestable, cambia cuando cambia cualquier cosa de la que depende). Con 0.83 el gestor está muy cerca del extremo inestable: como depende de tantas clases, cualquier cambio externo puede obligarlo a cambiar.

#### **Abstracción**

Donde  $m_a$  es el número de métodos abstractos y  $m_c$  el total de métodos de la clase.

$$A = \sum m_a / \sum m_c$$

$$A = 0 / 4$$

$$A = 0.00$$

**Interpretación:** sus cuatro métodos tienen implementación y no hay ninguna interfaz, así que la clase es completamente concreta. No existe ningún punto por donde extender su comportamiento sin modificarla.

#### **Distancia a la secuencia principal**

$$D = |A + I| - 1$$

$$D = |0.00 + 0.83| - 1$$

$$D = |-0.17|$$

$$D = 0.17$$

**Nota:** el enunciado escribe la fórmula como  $D = |A + I| - 1$ , pero así el resultado sale negativo (-0.17), y una distancia no puede ser negativa.

**Interpretación:** D mide qué tan lejos está la clase del equilibrio ideal entre abstracción y estabilidad. Un 0.17 parece un buen resultado, pero aquí engaña un poco: la clase está cerca de la línea solo porque es concreta e inestable al mismo tiempo, no porque esté bien diseñada. Su problema real es la baja cohesión que ya mostró en el ítem a.

#### **Posición en el plano A vs. I y Zona de Dolor**

La clase queda en el punto ( $I = 0.83$ ,  $A = 0.00$ ). La Zona de Dolor es la esquina ( $I = 0$ ,  $A = 0$ ), donde están las clases concretas de las que todo depende y que por eso son muy difíciles de

cambiar. El gestor por sí solo no cae ahí, porque es inestable.

Sin embargo, si se mide el módulo DentaCare.Legacy completo como un solo paquete, sí cae en la Zona de Dolor: Ca = 1 (Program), Ce = 0 (todas sus dependencias están dentro del mismo módulo). Es decir, el módulo es rígido: todo en él es concreto y no hay ninguna abstracción que permita cambiarlo o extenderlo sin tocar el código existente.

#### 4. Recálculo de métricas e impacto cuantitativo

Después de refactorizar, la lógica del gestor quedó repartida en tres servicios: AgendamientoCitasService, CancelacionCitasService y ReporteCitasService. Volví a calcular las métricas con los mismos criterios para compararlos con la clase original, y además medí cada capa del proyecto (Dominio, Aplicación e Infraestructura) como un paquete.

| Aspecto            | Métrica                                                | Antes (legado)                                          | Después (refactorizado)      | Qué cambió y por qué                                                                                                                                                                   |
|--------------------|--------------------------------------------------------|---------------------------------------------------------|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cohesión (SRP)     | LCOM96b de la clase principal                          | 0.50: cada atributo lo usa solo la mitad de los métodos | 0.00 en los tres servicios   | Cada servicio usa todos sus atributos porque hace una sola cosa. Separar responsabilidades eliminó la falta de cohesión.                                                               |
| Cohesión (SRP)     | Responsabilidades de la clase principal                | 11 en una sola clase                                    | 1 por servicio               | La God Class se repartió en clases pequeñas, cada una con un único motivo para cambiar.                                                                                                |
| Acoplamiento       | Ca                                                     | 1 (Program)                                             | 1 en cada servicio (Program) | No cambia: solo la raíz de composición crea los servicios.                                                                                                                             |
| Acoplamiento       | Ce                                                     | 5                                                       | 11, 8 y 2                    | Sube porque las dependencias ahora son explícitas y más pequeñas: cada servicio recibe por separado el validador, la calculadora, el repositorio, etc. Son más, pero de mejor calidad. |
| Acoplamiento (DIP) | Dependencias hacia clases concretas de infraestructura | 2 (SqlServerEjecutor y NotificacionServicio)            | 0                            | Los servicios dependen de interfaces. Cambiar SQL Server o el proveedor de correo ya no toca las reglas de negocio.                                                                    |
| Estabilidad        | I                                                      | 0.83                                                    | 0.92, 0.89 y                 | Ahora dependen de                                                                                                                                                                      |

|                       |                                                |                                     |                                                                        |                                                                                                                                                     |
|-----------------------|------------------------------------------------|-------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
|                       |                                                |                                     | 0.67                                                                   | interfaces estables y no de detalles que cambian seguido.                                                                                           |
| Abstracción           | A de la clase                                  | 0.00                                | 0.00                                                                   | Los servicios siguen siendo concretos; la abstracción ahora vive en las 13 interfaces que reciben.                                                  |
| Distancia             | D de la clase                                  | 0.17                                | 0.08, 0.11 y 0.33                                                      | Los dos servicios principales quedan más cerca de la secuencia principal. Reporte da 0.33 porque es una clase muy pequeña, con solo 2 dependencias. |
| Distancia del módulo  | D                                              | 1.00 (Zona de Dolor)                | 0.35 en promedio (Dominio 0.72, Aplicación 0.21, Infraestructura 0.13) | El sistema sale de la Zona de Dolor. Cada capa tiene un papel claro: el dominio es estable y la infraestructura es fácil de reemplazar.             |
| Abierto/cerrado (OCP) | Agregar una especialidad nueva                 | Modificar el if/else de AgendarCita | Crear 1 clase nueva                                                    | El código existente no se toca, así que no hay riesgo de romper lo que ya funciona.                                                                 |
| Segregación (ISP)     | Interfaces del módulo                          | 0                                   | 13                                                                     | Cada cliente depende solo del contrato que necesita.                                                                                                |
| Sustitución (LSP)     | Implementaciones intercambiables               | Ninguna                             | Repositorio SQL o en memoria; notificación por correo, SMS o consola   | Todas cumplen el mismo contrato y se pueden cambiar sin afectar a los servicios.                                                                    |
| Testabilidad (DIP)    | Probar sin base de datos ni servidor de correo | No es posible                       | Es posible                                                             | Se comprobó con 4 que dan los mismos resultados que el legado.                                                                                      |